

История языков программирования С и С++

С — процедурный язык программирования, разработанный в 1969—1973 годах сотрудниками Bell Labs Кеном Томпсоном и Деннисом Макалестэйр Ритчи как развитие языка Би. С был создан для использования в операционной системе UNIX, ключевым разработчиком которой являлся Деннис Ритчи. С тех пор он был перенесён на многие другие операционные системы и стал одним из самых используемых языков программирования.

Деннис Ритчи (9 сентября 1941 — 12 октября 2011) также известен как соавтор классической книги «Язык программирования С», обычно сокращаемой как «K&R» (авторы Керниган и Ритчи).

С++ — объектно-ориентированное расширение языка Си, создан в начале 1980-х годов Бьерном Страуструпом, также сотрудником Bell Labs. Вначале он добавил к языку Си возможность работы с классами и объектами, а в 1983 году после добавления большого количества синтаксических конструкций получившийся язык был переименован из Си с классами в «С++». Его первый коммерческий выпуск состоялся в октябре 1985 года.

С++ сочетает свойства как высокоуровневых, так и низкоуровневых языков. С момента создания он стал широко использоваться для разработки программного обеспечения, став одним из самых популярных языков программирования. На С++ создают операционные системы, прикладные программы, драйверы, компьютерные игры.

В 1985 году вышло первое издание книги «Язык программирования С++» Б. Страуструпа, которое стало классическим.

С++ оказал огромное влияние на другие языки программирования, в первую очередь на Java и С#.

Концепция языка С++

С++ — язык общего назначения и задуман для того, чтобы настоящие программисты получили удовольствие от самого процесса программирования.

Язык программирования С++ задумывался как язык, который будет:

- лучше и современней языка С;
- поддерживать абстракцию данных;
- поддерживать объектно-ориентированное программирование.
- содержать большую и расширяемую стандартную библиотеку.

За исключением второстепенных деталей, он практически содержит язык С как подмножество (хотя есть пример программы, которая является программой на языке С, но не может быть скомпилирована на языке С++). Язык С расширяется введением гибких и эффективных средств, предназначенных для построения новых типов. Программист структурирует свою задачу, определив новые типы, которые точно соответствуют понятиям предметной области задачи. Такой метод построения программы обычно называют абстракцией данных. Информация о типах содержится в некоторых объектах типов, определенных пользователем. С такими объектами можно работать надежно и просто даже в тех случаях, когда их тип нельзя установить на стадии трансляции. Программирование с использованием таких объектов обычно называют объектно-ориентированным. Если этот метод применяется правильно, то программы становятся короче и понятнее, а сопровождение их упрощается.

Ключевым понятием С++ является класс. Класс — это определяемый пользователем тип. Классы обеспечивают инкапсуляцию ("упрятывание") данных, их инициализацию, неявное преобразование пользовательских типов, динамическое задание типов, контролируемое пользователем управление памятью и средства для

перегрузки операций. В языке C++ концепции контроля типов и модульного построения программ реализованы более полно, чем в C. Кроме того, C++ содержит усовершенствования, прямо с классами не связанные: символические константы, функции-подстановки, стандартные значения параметров функций, перегрузка имен функций, операции управления свободной памятью и ссылочный тип. В C++ сохранены все возможности C эффективной работы с основными объектами, отражающими аппаратную "реальность" (разряды, байты, слова, адреса и т.д.). Это позволяет достаточно эффективно реализовывать пользовательские типы.

Как язык, так и стандартные библиотеки C++ проектировались в расчете на переносимость. Имеющиеся реализации языка будут работать в большинстве систем, поддерживающих C. В программах на C++ можно использовать библиотеки C. Большинство служебных программ, рассчитанных на C, можно использовать и в C++.

Можно сказать, что Си и C++ сосуществуют между собой. Когда в 2011 году вышел новый стандарт языка C++ — C++11, вместе с ним вышел и стандарт языка Си — C11.

Заголовочные файлы стандартной библиотеки C++

Все заголовочные файлы стандартной библиотеки языка C++ не содержат расширения .h. Например:

```
#include<iostream>
#include<vector>
#include<algorithm>
```

Заголовочные файлы из стандартной библиотеки языка C можно использовать в языке C++, но их имена изменились - в начало файла добавилась буква "c", а расширение ".h" исчезло. То есть при желании использовать функции, которые в языке C определены в заголовочных файлах stdio.h или math.h, их требуется подключать следующим образом:

```
#include<cstdio>
#include<cmath>
```

Комментарии в тексте программы

(001_comments.cpp)

Комментарий — это заметка, читаемая программистом, которая вставляется непосредственно в исходный код программы. Компилятор игнорирует комментарии, и они предназначены исключительно для использования программистом.

В языке программирования C++ существует два различных стиля комментариев, оба из которых служат одной цели: помогают программистам документировать код определенным образом.

В языке C допускались только многострочные комментарии. Начало комментария обозначалось символами /*, конец - символами */.

Пример:

```
/* Это комментарий.
   он может занимать несколько строк */
```

Поскольку всё, что находится между символами, игнорируется, вы иногда можете видеть, как программисты «украшают» свои многострочные комментарии:

```
/* This is a multi-line comment.
 * the matching asterisks to the left
 * can make this easier to read
 */
```

В языке C++ появились однострочные комментарии - они отмечаются символами // и продолжаются до конца строки:

```
int n; // Размер считываемого массива
```

Не используйте многострочные комментарии внутри других многострочных комментариев. Можно использовать однострочные комментарии внутри многострочных.

```
std::cout << "Hello world!\n"; // std::cout lives in the iostream library
std::cout << "It is very nice to meet you!\n"; // these comments make the code hard to read
std::cout << "Yeah!\n"; // especially when lines are different lengths
```

Если комментарии находятся справа от строки, их может быть трудно читать как в коде, так и в комментариях, особенно если строка длинная. Если строки довольно короткие, комментарии можно просто выровнять (обычно по табуляции), например так:

```
std::cout << "Hello world!\n";           // std::cout lives in the iostream library
std::cout << "It is very nice to meet you!\n"; // this is much easier to read
std::cout << "Yeah!\n";                 // don't you think so?
```

Комментарии для документирования кода выглядят следующим образом:

```
/**
 * Точка входа в приложение
 * @return Результат работы программы, в виде числа
 */
int main() {
}
```

Стандартная библиотека C++

Язык C++ содержит обширную стандартную библиотеку (STL).

Основные части библиотеки следующие:

1. Ввод-вывод описан в заголовочных файлах `iostream`, `fstream` и других.
2. Работа со строками описана в файле `string` и других.
3. Контейнеры (структуры данных) описаны в большом числе заголовочных файлов в соответствии с типом контейнера. Например,
`vector` - динамический массив,
`set` - множество с возможностью быстрого добавления, удаления, поиска элементов,
`map` - ассоциативный массив (словарь),
`list` - двусвязный список,
`stack` - стек,
`queue` - очередь.
4. Алгоритмы (линейный и двоичный поиск, нахождение следующей перестановки, случайная перестановка) описаны в заголовочном файле `algorithm`.

Первоначально часть стандартной библиотеки называлась STL - Standard Template Library и развивалась независимо от языка C++ компаниями HP и SGI. Затем она была добавлена в стандарт языка, но название STL сохранилось и часто употребляется применительно к той части библиотеки, которая относится к контейнерам и алгоритмам.

Пространства имен

(002_namespaces1.cpp)

Имена (функций, переменных) в языке C++ можно разделять на "пространства имен" для удобства - чтобы могли существовать функции и переменные с одинаковыми именами в разных "пространствах имен".

Допустим, вы впервые едете в гости к другу, и вам дали адрес: Фронт-стрит, 245, Милл-Сити. Добравшись до Милл-Сити, вы достаёте карту и обнаруживаете, что в Милл-Сити на самом деле есть две Фронт-стрит, расположенные в разных городах! В какой из них вы поедете? Если бы у вас не было дополнительной подсказки (например, вы помните, что дом вашего друга находится у реки), вам пришлось бы позвонить другу и попросить больше информации. Поскольку это было бы запутанно и неэффективно (особенно для вашего почтальона), в большинстве стран все названия улиц и адреса домов в пределах города должны быть уникальными.

Аналогичным образом C++ требует, чтобы все идентификаторы были однозначными. Если в одной программе используются два одинаковых идентификатора таким образом, что компилятор или компоновщик не могут их различить, компилятор или компоновщик выдаст ошибку. Эта ошибка обычно называется конфликтом имён (или совпадением имён).

Если конфликтующие идентификаторы находятся в одном и том же файле, результатом будет ошибка компилятора. Если конфликтующие идентификаторы находятся в отдельных файлах, относящихся к одной и той же программе, результатом будет ошибка компоновщика.

Пространство имен объявляется так:

```
namespace my_namespace
{
    // Описание функций, переменных, классов
    int var;
};
```

Для доступа к переменной `var` из пространства имен `my_namespace` нужно писать `my_namespace::var`.

Можно также использовать инструкцию:

```
using namespace my_namespace;
```

тогда все имена из пространства имен `my_namespace` можно использовать без указания имени пространства имен (просто писать `var`).

Пример коллизии именований:

a.cpp:

```
#include <iostream>

void myFcn(int x)
{
    std::cout << x;
}
```

main.cpp:

```
#include <iostream>

void myFcn(int x)
{
    std::cout << 2 * x;
}

int main()
{
    return 0;
}
```

Когда компилятор скомпилирует эту программу, он скомпилирует `a.cpp` и `main.cpp` независимо друг от друга, и каждый файл скомпилируется без проблем.

Однако при выполнении компоновщика все определения в `a.cpp` и `main.cpp` будут связаны между собой, и будут обнаружены конфликтующие определения для функции `myFcn()`. После этого компоновщик прервёт работу с ошибкой. Обратите внимание, что эта ошибка возникает, даже если `myFcn()` никогда не вызывается!

Большинство коллизий именования происходят в двух случаях:

1. Две (или более) функции (или глобальные переменные) с одинаковыми именами вводятся в отдельные файлы, принадлежащие одной и той же программе. Это приведёт к ошибке компоновщика, как показано выше.
2. В один и тот же файл вводятся две (или более) функции (или глобальных переменных) с одинаковыми именами. Это приведёт к ошибке компиляции.

По мере того, как программы становятся больше и используют больше идентификаторов, вероятность возникновения коллизий имён значительно возрастает. Хорошая новость в том, что C++ предоставляет множество механизмов для предотвращения коллизий имён. Одним из таких механизмов является локальная область видимости, которая не позволяет локальным переменным, определённым внутри функций, конфликтовать друг с другом. Но локальная область видимости не работает для имён функций. Как же предотвратить конфликты имён функций?

Область видимости — это область исходного кода, в которой все объявленные идентификаторы считаются отличными от имён, объявленных в других областях видимости (как города в нашей аналогии). Два идентификатора с одинаковыми именами могут быть объявлены в разных областях видимости без возникновения конфликта имён. Однако в пределах одной области видимости все идентификаторы должны быть уникальными, иначе возникнет конфликт имён.

В примере в начале урока функции `main()` и обе версии `myFcn()` определены в глобальном пространстве имён. Конфликт имён, возникший в этом примере, происходит из-за того, что обе версии `myFcn()` оказываются в глобальном пространстве имён, что нарушает правило, согласно которому все имена в области видимости должны быть уникальными.

Предупреждение

Не используйте директивы (например, `using namespace std;`) в начале программы или в заголовочных файлах. Они нарушают принцип, по которому были добавлены пространства имён.

Стандартная библиотека

Вся стандартная библиотека находится в пространстве имен **std**, поэтому нужно либо писать

```
std::cout << a << std::endl;
```

для вывода переменной `a`. Или написать в начале программы

```
using namespace std;
```

и тогда можно будет просто писать

```
cout << a << endl;
```

Ввод-вывод в языке C++

Для стандартного ввода-вывода в языке C++ используется заголовочный файл **iostream**.

В нем объявлены объекты **cin** для ввода с клавиатуры и **cout** для вывода на экран.

Чтобы считать со стандартного ввода значения переменных `a`, `b`, `c`, нужно написать:

```
std::cin >> a >> b >> c;
```

Для вывода на экран этих переменных нужно написать:

```
std::cout << a << b << c;
```

Для разделения значения переменных пробелами, нужно выводить строку из одного пробела между ними:

```
std::cout << a << " " << b << " " << c;
```

Типы переменных определяются автоматически благодаря технологии перегрузки операторов.

Чтобы вывести конец строки нужно вывести стандартный объект endl:

```
std::cout << std::endl;
```

Можно не писать **std::**, если дать инструкцию **using namespace std** в начале программы.

Инициализация переменных

(003_initialization.cpp)

Короткая программа, которая сначала выделяет одну целочисленную переменную с именем x, а затем выделяет ещё две целочисленные переменные с именами y и z:

```
int main()
{
    int x;    // define an integer variable named x
    int y, z; // define two integer variables, named y and z

    return 0;
}
```

После определения переменной вы можете присвоить ей значение (в отдельном операторе) с помощью оператора =. Этот процесс называется присвоением, а оператор = называется оператором присвоения.

```
int width; // define an integer variable named width
width = 5; // assignment of value 5 into variable width

// variable width now has value 5
```

По умолчанию при присваивании значение из правой части оператора = копируется в переменную из левой части оператора. Это называется присваиванием с копированием.

Как только переменной присваивается значение, его можно вывести с помощью std::cout и оператора <<.

Предупреждение

Одна из самых распространённых ошибок, которые допускают начинающие программисты, — это путаница между оператором присваивания (=) и оператором равенства (==). Оператор присваивания (=) используется для присвоения значения переменной. Оператор равенства (==) используется для проверки, равны ли два операнда по значению.

Недостатком присваивания является то, что для присвоения значения только что определённому объекту требуется два оператора: один для определения переменной, а другой для присвоения значения.

Эти два шага можно объединить. При определении объекта можно дополнительно указать начальное значение для объекта. Процесс указания начального значения для объекта называется инициализацией, а синтаксис, используемый для инициализации объекта, называется инициализатором. Неофициально начальное значение также часто называют «инициализатором».

Такой синтаксис был добавлен в стандарте C++11 и называется инициализацией фигурных скобок (также иногда называемой uniform инициализацией, инициализация прямым списком).

Есть две формы такой инициализации:

1. Прямая. Например: `int width{ 5 };` — прямая инициализация значения 5 в переменную `width`. 1
2. Инициализация копирования. Пример: `int height = { 6 };` — инициализация копирования значения 6 в переменную `height`. 1

Инициализация переменной с пустыми фигурными скобками указывает на нулевую инициализацию. Обычно в таком случае переменная инициализируется до нуля (или пустой, если это более подходит для данного типа). Пример: `int width {};` — нулевая инициализация до значения 0. 1

Инициализация фигурными скобками безопаснее, так как запрещает «сужающие» преобразования. Если попытаться использовать такую инициализацию для переменной со значением, которое она не может безопасно удерживать, компилятор выдаст предупреждение или ошибку.

В C++ существует 5 распространенных форм инициализации:

```
int a;           // default-initialization (no initializer)

// Traditional initialization forms:
int b = 5;        // copy-initialization (initial value after equals sign)
int c ( 6 );      // direct-initialization (initial value in parenthesis)

// Modern initialization forms (preferred):
int d { 7 };      // direct-list-initialization (initial value in braces)
int e {};         // value-initialization (empty braces)
```

Наилучшая практика

Для инициализации переменных используйте инициализацию прямым списком или инициализацию значением.

Константы

Объявления констант

(004_ocnstants.cpp)

В языке C для объявления констант использовались директивы препроцессора `#define`. Например:

```
#define N 100
```

Это низкоуровневый и опасный механизм. Например, объявленную таким образом константу на самом деле можно переопределить:

```
#define N 1000
```

В языке C++ появились константные выражения, которые нужно использовать вместо `#define`

В программировании константа — это значение, которое нельзя изменить во время выполнения программы.

C++ поддерживает два различных типа констант:

- Именованные константы — это постоянные значения, связанные с идентификатором. Их также иногда называют символическими константами.
- Литеральные константы - это постоянные значения, которые не связаны с идентификатором.

Чтобы объявить константную переменную, мы помещаем ключевое слово `const` (называемое «квалификатором `const`») рядом с типом объекта.

Константные переменные должны быть инициализированы при их определении, и затем это значение нельзя изменить с помощью присваивания:

```
const int N = 100;
const double gravity { 9.8 }; // preferred use of const before type
int const sidesInSquare { 4 }; // "east const" style, okay but not preferred
```

Наилучшая практика

Поместите const перед типом (потому что так принято делать).

Обратите внимание, что константные переменные могут быть инициализированы из других переменных (в том числе неконстантных):

```
#include <iostream>

int main()
{
    std::cout << "Enter your age: ";
    int age{};
    std::cin >> age;

    const int constAge { age }; // initialize const variable using non-const value

    age = 5; // ok: age is non-const, so we can change its value
    constAge = 6; // error: constAge is const, so we cannot change its value

    return 0;
}
```

Существует ряд различных соглашений об именовании, которые используются для переменных const.

Программисты, перешедшие с C, часто предпочитают имена константных переменных с подчёркиванием и заглавной буквы (например, EARTH_GRAVITY). В C++ чаще используются имена с заглавной буквы и префиксом «k» (например, kEarthGravity).

Однако, поскольку константные переменные ведут себя как обычные переменные (за исключением того, что им нельзя присваивать значения), нет причин использовать для них специальное соглашение об именовании. По этой причине мы предпочитаем использовать то же соглашение об именовании, что и для неконстантных переменных (например, earthGravity).

Параметры функции можно сделать постоянными с помощью ключевого слова const:

```
#include <iostream>

void printInt(const int x)
{
    std::cout << x << '\n';
}

int main()
{
    printInt(5); // 5 will be used as the initializer for x
    printInt(6); // 6 will be used as the initializer for x

    return 0;
}
```

Если сделать параметр функции константой, компилятор поможет убедиться, что значение параметра не изменится внутри функции. Однако в современном C++ мы не делаем параметры-значения, const потому что нам, как правило, всё равно, если функция изменит значение параметра (поскольку это всего лишь копия, которая в любом случае будет уничтожена в конце функции). Ключевое слово const также добавляет немного лишнего в прототип функции.

Не используйте const для определения параметров значения.

Если переменную можно сделать константой, то, как правило, её следует сделать константой. Это важно по ряду причин:

- Это снижает вероятность возникновения ошибок. Сделав переменную константой, вы гарантируете, что значение не будет изменено случайно.
- Это даёт компилятору больше возможностей для оптимизации программ. Когда компилятор может предположить, что значение не меняется, он может использовать больше методов для оптимизации программы, в результате чего скомпилированная программа будет меньше и быстрее. Мы обсудим это подробнее далее в этой главе.
- Самое главное, что это снижает общую сложность наших программ. Когда мы пытаемся определить, что делает фрагмент кода, или отлаживаем проблему, мы знаем, что значение константной переменной не может быть изменено, поэтому нам не нужно беспокоиться о том, меняется ли её значение, к какому значению оно меняется и является ли это новое значение правильным.

Кроме того, слово `const` используется при описании параметров функции вместе со ссылками для объявления "константных ссылок" - объект передается в функцию по ссылке, а не по значению (то есть не создается копия объекта, что важно для больших контейнеров, копирование которых может занимать много времени), но при этом функция не может модифицировать передаваемый объект.

Константы времени компиляции

Мы можем обратиться за помощью к компилятору, чтобы получить константную переменную времени компиляции там, где нам это нужно. Для этого мы используем ключевое слово `constexpr` (сокращение от «константное выражение») вместо `const` в объявлении переменной. Переменная `constexpr` всегда является константой времени компиляции. В результате переменная `constexpr` должна быть инициализирована константным выражением, иначе возникнет ошибка компиляции.

```
#include <iostream>

// The return value of a non-constexpr function is not constexpr
int five()
{
    return 5;
}

int main()
{
    constexpr double gravity { 9.8 }; // ok: 9.8 is a constant expression
    constexpr int sum { 4 + 5 };      // ok: 4 + 5 is a constant expression
    constexpr int something { sum };  // ok: sum is a constant expression

    std::cout << "Enter your age: ";
    int age{};
    std::cin >> age;

    constexpr int myAge { age };      // compile error: age is not a constant expression
    constexpr int f { five() };       // compile error: return value of five() is not constexpr

    return 0;
}
```

Кроме того, `constexpr` работает для переменных с нецелыми типами:

```
constexpr double d { 1.2 }; // d can be used in constant expressions!
```

Значение `const` по сравнению с `constexpr` для переменных

Для переменных:

const Это означает, что значение объекта нельзя изменить после инициализации. Значение инициализатора может быть известно во время компиляции или выполнения. Значение объекта `const` может быть вычислено во время выполнения.

constexpr Это означает, что объект можно использовать в константном выражении. Значение инициализатора должно быть известно во время компиляции. Объект `constexpr` может быть вычислен во время выполнения или во время компиляции.

Переменные `constexpr` неявно являются константными. Константные переменные неявно не являются переменными `constexpr` (за исключением константных целочисленных переменных с инициализатором в виде константного выражения). Хотя переменная может быть определена как `constexpr` и `const`, в большинстве случаев это избыточно, и нам нужно использовать только `const` или `constexpr`.

Наилучшая практика

Любая константная переменная, инициализатором которой является константное выражение, должна быть объявлена как `constexpr`.

Любая константная переменная, инициализатор которой не является константным выражением (что делает её константой во время выполнения), должна быть объявлена как `const`.

Ссылки

(004_references.cpp)

Вместо концепции указателей в языке C++ появились более удобные и безопасные ссылки. Как правило, их используют для передачи объекта в функцию, без использования указателей, но с возможностью модификации объекта по ссылке.

Ссылку можно понимать как безопасный вариант указателя. При этом ссылки имеют особенности, отличающие их от указателей:

1. При объявлении ссылка обязательно на уже существующий объект данного типа. Ссылка не может ссылаться "ни на что".
2. Ссылка от её объявления до её исчезновения указывает на один и тот же адрес.
3. При обращении к ссылке разыменование происходит автоматически.
4. Адрес ссылки — это адрес исходного объекта, на который она указывает.

Объявление ссылок очень похоже на объявление указателей, только вместо звёздочки `*` пишется амперсанд `&`.

При объявлении ссылка обязана быть инициализирована.

```
int &x; // недопустимо!
int &x = veryLongVariableName; // допустимо. Теперь x - это альтернативное имя переменной
veryLongVariableName
int A[10];
int &x = A[5]; // Ссылка может указывать на элемент массива
x++;          // то же, что A[5]++;
x = 1;        // то же, что A[5] = 1;

int x { 5 }; // normal integer variable
int& ref { x }; // ref is now an alias for variable x

std::cout << x << ref << '\n'; // print 55

x = 6; // x now has value 6

std::cout << x << ref << '\n'; // prints 66

ref = 7; // the object being referenced (x) now has value 7

std::cout << x << ref << '\n'; // prints 77
```

После инициализации ссылка в C++ не может быть повторно инициализирована, то есть её нельзя изменить, чтобы она ссылалась на другой объект.

Передача параметров в функцию по ссылке

Параметры можно передавать по ссылкам. При этом связывание ссылки с определённой переменной произойдёт в момент вызова функции (на этапе выполнения программы).

```
void foo(int &x)
{
    x = 3;
}
int main()
{
    int t = 1;
    foo (t);
    cout << t; // выведет 3
}
```

Нужно быть осторожным при передаче значений в такую функцию стороннего разработчика. Если не планируется изменение передаваемой переменной, функцию лучше объявить так:

```
void foo(const int &x);
```

Это гарантирует программисту-пользователю функции неизменность передаваемого значения.

Для типа **int** передача по константной ссылке обычно не нужна, так как можно просто передать аргумент по значению, но для большого класса или структуры передача по ссылке гораздо быстрее и экономит память.

Чего нельзя делать со ссылкой

Возврат из функции ссылки на автоматически созданный объект (локальную переменную) приводит к появлению "битых ссылок", значение которых непредсказуемо.

Также синтаксис C++ не позволяет создавать указатели на ссылки и массивы ссылок.

Ссылки и референты имеют независимое время жизни

Время жизни ссылки и время жизни её референта не зависят друг от друга. Другими словами, верны оба утверждения:

- Ссылка может быть уничтожена раньше объекта, на который она ссылается.
- Объект, на который делается ссылка, может быть уничтожен до создания ссылки.

Когда ссылка уничтожается раньше, чем объект, на который она ссылается, это не влияет на объект. Следующая программа демонстрирует это:

```
#include <iostream>

int main()
{
    int x { 5 };

    {
        int& ref { x }; // ref is a reference to x
        std::cout << ref << '\n'; // prints value of ref (5)
    } // ref is destroyed here -- x is unaware of this

    std::cout << x << '\n'; // prints value of x (5)

    return 0;
} // x destroyed here
```

Логический тип данных

(006_bool.cpp)

В языке C++ для логических значений существует специальный тип — **bool**.

Допустимыми значениями этого типа являются только **true** и **false**, при этом других значений у переменной данного типа быть не может.

Переменная типа **bool** занимает в памяти ровно 1 байт.

```
bool b;  
cout << sizeof(b); //выведет 1
```

Допустимые операции

С логическими переменными можно осуществлять логические операции:

~x — логическое отрицание (НЕ)

x & y — логическое умножение (И)

x | y — логическое сложение (ИЛИ)

Для типа **bool** стираются различия между операторами **&&** и **&**, а также между **||** и **|**.

Совместимость с типом **int**

Тип **bool** совместим с типом **int** по присваиванию в обе стороны.

При этом **true** переходит в 1, **false** — в 0.

При обратном приведении любое число, не равное нулю — переходит в **true**, 0 — в **false**.

Если использовать **bool** в арифметическом выражении, то оно будет переведено в **int**: **bool + bool = int**.

Надо понимать, что в C++ логический и целочисленный тип — это разные типы, поэтому по типу аргумента **int** и **bool** возможна перегрузка функций.

Значения по умолчанию для параметров функций

(007_func.cpp)

При объявлении функции можно назначить значение аргумента "по умолчанию". Если этот аргумент не будет задан, то функция будет использовать значение "по умолчанию".

Например:

```
int factorial(int n=0)
```

Если вызывать функцию без параметров, то значение параметра **n** будет равно 0.

"По умолчанию" можно передавать только последние параметры функции. Например, объявим функцию так:

```
void f(int x, int y=1, int z=2);
```

Эту функцию можно вызвать от одного, двух, или трех параметров. Если вызвать ее от одного параметра, то будет передано значение **x**, а значения **y** и **z** возьмутся "по умолчанию" и будут равны 1 и 2. Если вызвать от 2 параметров - то будут заданы значения **x** и **y**, а значение **z** будет равно 2.

Работа с динамической памятью

Для работы с динамической памятью вместо функций **malloc** и **free** языка C в языке C++ введены операторы **new** и **delete**. Использование функций языка C для работы с динамической памятью не рекомендуется в языке C++

Работа со строками символов

Рассмотрим следующий пример программы:

```
#include <iostream>

int main()
{
    std::cout << "Hello, world!"; // "Hello world!" is a C-style string literal.
    return 0;
}
```

Строковые литералы в стиле C можно использовать без проблем, строковые переменные в стиле C ведут себя странно, с ними трудно работать (например, нельзя использовать присваивание, чтобы присвоить строковой переменной в стиле C новое значение), и они опасны (например, если вы скопируете большую строку в стиле C в пространство, выделенное для более короткой строки в стиле C, это приведёт к неопределённому поведению). В современном C++ лучше избегать строковых переменных в стиле C.

В C++ появились два дополнительных строковых типа, с которыми гораздо проще и безопаснее работать: `std::string` и `std::string_view`

Самый простой способ работать со строками и строковыми объектами в C++ - это использовать `std::string` тип, который находится в заголовке `<string>`.

Мы можем создавать объекты типа `std::string` точно так же, как и другие объекты:

```
#include <string> // allows use of std::string

int main()
{
    std::string name {}; // empty string

    return 0;
}
```

Как и обычные переменные, вы можете инициализировать или присваивать значения объектам `std::string`, как и ожидалось:

```
#include <string>

int main()
{
    std::string name { "Alex" }; // initialize name with string literal "Alex"
    name = "John";              // change name to "John"

    return 0;
}
```

Вывод строки с помощью `std::cout`

`std::string` объекты могут быть выведены как ожидалось с помощью `std::cout`:

```
#include <iostream>
#include <string>

int main()
{
    std::string name { "Alex" };
    std::cout << "My name is: " << name << '\n';

    return 0;
}
```

На консоль будет выведено:

```
My name is: John
```

`std::string` может обрабатывать строки разной длины

Одна из самых полезных функций `std::string` — хранение строк разной длины:

```
#include <iostream>
#include <string>

int main()
{
    std::string name { "Alex" }; // initialize name with string literal "Alex"
    std::cout << "My name is: " << name << '\n';

    name = "Jason";              // change name to a longer string
    std::cout << "My name is: " << name << '\n';

    name = "Jay";                // change name to a shorter string
    std::cout << "My name is: " << name << '\n';

    return 0;
}
```

На консоль будет выведено:

```
My name is: John
My name is: Jason
My name is: Jay
```

Ключевое понимание

Если `std::string` не хватает памяти для хранения строки, он запросит дополнительную память (во время выполнения), используя динамическое выделение памяти. Эта возможность получать дополнительную память — одна из причин, почему `std::string` такой гибкий, но при этом сравнительно медленный.

Ввод строки с помощью `std::cin`

Использование `std::string` с `std::cin` может преподнести некоторые сюрпризы! Рассмотрим следующий пример:

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Enter your full name: ";
    std::string name{};
    std::cin >> name; // this won't work as expected since std::cin breaks on whitespace

    std::cout << "Enter your favorite color: ";
    std::string color{};
    std::cin >> color;

    std::cout << "Your name is " << name << " and your favorite color is " << color << '\n';

    return 0;
}
```

Вот результаты примерного запуска этой программы:

```
Enter your full name: Джон Доу
Enter your favorite color: Your name is Джон and your favorite color is Доу
```

При использовании оператора `>>` для извлечения строки из `std::cin` оператор `>>` возвращает только символы до первого пробела, который он встречает. Все остальные символы остаются внутри `std::cin` в ожидании следующего извлечения.

Поэтому, когда мы использовали оператор `>>` для извлечения входных данных в переменную `name`, извлекался только "John", оставляя " Doe" внутри `std::cin`. Когда мы затем использовали оператор `>>` для извлечения входных данных в переменную `color`, она извлекалась "Doe" вместо того, чтобы ждать, пока мы введем цвет. Затем программа завершается.

Используйте `std::getline()` для ввода текста

Чтобы преобразовать полную строку входных данных в строку, вам лучше использовать вместо этого функцию `std::getline()`. `std::getline()` требует два аргумента: первый - `std::cin`, а второй - ваша строковая переменная.

Вот та же программа, что и выше, использующая `std::getline()`:

```
#include <iostream>
#include <string> // For std::string and std::getline

int main()
{
    std::cout << "Enter your full name: ";
    std::string name{};
    std::getline(std::cin >> std::ws, name); // read a full line of text into name

    std::cout << "Enter your favorite color: ";
    std::string color{};
    std::getline(std::cin >> std::ws, color); // read a full line of text into color

    std::cout << "Your name is " << name << " and your favorite color is " << color << '\n';

    return 0;
}
```

Теперь наша программа работает так, как ожидалось:

```
Enter your full name:Джон Доу
Enter your favorite color:синий
Your name is Джон Доу and your favorite color is синий
```

`std::ws`

C++ также поддерживает манипуляторы ввода, которые изменяют способ приема входных данных. `std::ws` Манипулятор ввода сообщает `std::cin` игнорировать любые начальные пробелы перед извлечением. Начальный пробел - это любой пробельный символ (пробелы, табуляция, перевод строки), который встречается в начале строки.

Давайте разберемся, почему это полезно. Рассмотрим следующую программу:

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Pick 1 or 2: ";
    int choice{};
    std::cin >> choice;

    std::cout << "Now enter your name: ";
    std::string name{};
    std::getline(std::cin, name); // note: no std::ws here

    std::cout << "Hello, " << name << ", you picked " << choice << '\n';

    return 0;
}
```

Вот некоторые выходные данные этой программы:

```
Выберите 1 или 2: 2
Теперь введите своё имя: Здравствуйте, вы выбрали 2
```

Эта программа сначала попросит вас ввести 1 или 2 и будет ждать, пока вы это сделаете. Пока всё хорошо. Затем она попросит вас ввести ваше имя. Однако она не будет ждать, пока вы введёте имя! Вместо этого она выведет строку «Привет» и завершит работу.

Когда вы вводите значение с помощью `operator>>`, `std::cin` не только сохраняет значение, но и сохраняет символ новой строки (`'\n'`), который появляется, когда вы нажимаете клавишу Enter. Поэтому, когда мы вводим 2 и нажимаем Enter, `std::cin` сохраняет строку `"2\n"` в качестве входных данных. Затем он извлекает значение 2 в переменную `choice`, оставляя символ новой строки на потом. Затем, когда `std::getline()` извлекает текст в `name`, он видит, что `"\n"` уже находится в `std::cin`, и предполагает, что мы, должно быть, ранее ввели пустую строку! Это точно не то, что мы хотели сделать.

Мы можем изменить приведённую выше программу, чтобы использовать манипулятор ввода `std::ws` и указать `std::getline()` игнорировать любые ведущие пробелы:

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Pick 1 or 2: ";
    int choice{};
    std::cin >> choice;

    std::cout << "Now enter your name: ";
    std::string name{};
    std::getline(std::cin >> std::ws, name); // note: added std::ws here

    std::cout << "Hello, " << name << ", you picked " << choice << '\n';

    return 0;
}
```

Теперь эта программа будет функционировать по назначению:

```
Pick 1 or 2:2
Now enter your name: Алекс
Hello, Алекс, you picked 2
```

Наилучшая практика

Если вы используете `std::getline()` для чтения строк, используйте `std::cin >> std::ws` для ввода, чтобы игнорировать ведущие пробелы. Это нужно делать для каждого вызова `std::getline()`, так как `std::ws` не сохраняется между вызовами.

Ввод-вывод в C++

Какое главное видимое отличие вывода на экран и ввода с клавиатуры в Си и C++? В Си они были синтаксически оформлены как вызов функции, а в C++ — это **операция** над объектом специального типа — **поток**ом.

Потоки определяются в библиотеке **`iostream`**, где определен вывод для каждого встроенного типа.

Вывод

(008_cout.cpp)

Все идентификаторы стандартной библиотеки шаблонов STL (*Standard Template Library*) определены в пространстве имен **`std`**, что означает необходимость обращения к ним через квалификатор **`std::`**:

```
std::cout << 100;
std::cout << 'ege';
std::cout << '.';
std::cout << true;
std::cout << std::endl;
```

Заметим, что в C++ мы не прописываем типы выводимых значений, компилятор неким (пока непонятным) способом разбирается в типе выводимого значения, и выводит его соответствующим образом.

Вывод в один и тот же поток можно писать в одну строчку:

```
std::cout << 100 << 'ege' << '.' << true << std::endl;
```

Для вывода в поток ошибок определён поток `cerr`.

Управление выводом на консоль для различных кодировок

Рассмотрим следующий пример, который должен вывести на консоль/терминал строку:

```
#include <iostream>

int main()
{
    std::cout << "Hello, World! Привет, мир! 你好, 世界! ?\n";
    return 0;
}
```

Результатом будет:

```
Hello, World! ㄚятАㄚ ㄚㄚㄚㄚтВ, ㄚㄚㄚтА! фӀахеӀ, фӀцчХМ! ?
```

Дело в том, что действующий стандарт языка C++ (ISO/IEC 14882:2020) определяет для исходного кода (текста) программы базовый набор возможных символов (basic source character set) в количестве 96 штук (в том числе буквы латиницы), в который можно добавлять дополнительные символы из набора символов Юникода с помощью специальной нотации (universal character name).

Также свои сложности вносит особенности работы консоли (в частности Windows), у которой есть своя кодовая страница для вывода символов.

Чтобы принудительно переключить кодировку на консоли можно использовать два способа:

1. Если программа будет работать только под операционной системой Windows, то можно использовать следующий вариант:

```
#include <windows.h> // для функции SetConsoleOutputCP и константы CP_UTF8
#include <iostream>

int main()
{
    SetConsoleOutputCP(CP_UTF8);

    std::cout << "Hello, World! Привет, мир! 你好, 世界! ?\n";
    return 0;
}
```

2. Использование возможностей языка C++ для переключения режимов работы с потоками

```
#include <io.h> // для функции _setmode
#include <fcntl.h> // для константы _O_U8TEXT
#include <iostream>

int main()
{
    _setmode(_fileno(stdout), _O_U8TEXT);

    std::wcout << L"Hello, World! Привет, мир! 你好, 世界! ?\n";
    return 0;
}
```

В исходный файл потребовалось подключить два заголовочных файла для вызова функции изменения режима работы потока `_setmode`, а также для использования константы переключения режима.

Также была заменена функция `std::cout` на `std::wcout`, а также добавлена приставка «L» к строковому литералу. Эти два последних изменения нужны для работы с так называемыми «широкими» символами (тип «`wchar_t`»).

В итоге, строка будет выводиться корректно:

```
Hello, World! Привет, мир! 你好, 世界! ?
```

Вывод в строку

Для вывода в строку можно использовать класс `stringstream` / `wstringstream`;

Принцип работы аналогичен использованию `cout`

Для получения содержимого в виде строки нужно использовать метод `str` без параметров.

Чтобы очистить поток от содержимого, также можно использовать метод `str`, но с параметром в виде пустой строки:

```
std::stringstream buffer;
buffer << "String Stream: Hello, World!";
std::string str = buffer.str();
std::cout << str << std::endl;
buffer.str("");
```

Ввод

(008_cin.cpp)

Поток ввода с клавиатуры называется **cin**, а считывание из потока производится другой операцией `>>` :

```
std::cin >> x;
```

Тип считываемого значения определяется автоматически по типу переменной `x`.

Для всех типов, кроме `char`, считывание будет производиться с пропуском символов-разделителей и до следующего символа-разделителя. При этом пробел и табуляция так же как и символ перевода каретки являются корректными разделителями. Считывание в `char` происходит посимвольно независимо от типа символа.

Например для введенной строки "Иван Иванович Иванов",

```
std::string name;
std::cin >> name;
```

считает в `name` только первое слово "Иван".

Считать всю строку целиком можно с помощью функции **`getline()`**:

```
std::string name;
std::getline(std::cin, name);
```

Считывать несколько значений можно также в одну строку:

```
std::cin >> x >> y >> z;
```

Форматированный ввод-вывод в C++

Для управления вводом-выводом в C++ используются:

- флаги форматного ввода-вывода;
- манипуляторы форматирования.

Использование флагов форматного ввода-вывода

Флаги позволяют включить или выключить один из параметров вывода на экран. Для установки флага вывода используется следующая конструкция языка C++:

```
cout.setf( ios::flag)
```

Для снятия флага применяют конструкцию

```
cout.unsetf( ios::flag)
```

здесь flag — имя конкретного флага.

Если при выводе необходимо установить несколько флагов, то можно воспользоваться арифметической операцией "или" (|). В этом случае конструкция языка C++ будет такой:

```
cout.setf( ios::flag1|ios::flag2|ios::flag3)
```

В данном случае flag1, flag2, flag3 — имена устанавливаемых флагов вывода.

В следующей приведены некоторые флаги форматного вывода с примерами их использования.

Флаг	Описание	Пример использования ¹	Результат
right	Выравнивание по правой границе	int r=-25; cout.setf(ios::right); cout.width(15); cout<<"r="<<r<<endl;	r=-25
left	Выравнивание по левой границе (по умолчанию)	double r=-25.45; cout.setf(ios::left); cout.width(50); cout<<"r="<<r<<endl;	r=-25.45
boolalpha	Вывод логических величин в текстовом виде (true, false)	bool a=true; cout<<a<<endl; cout.setf(ios::boolalpha); cout<<a<<endl;	true
dec	Вывод величин в десятичной системе счисления (по умолчанию)	int r=-25; cout<<"r="<<r<<endl;	r=-25
oct	Вывод величин в восьмеричной системе счисления	int p=23; //Отменить, установленный по умолчанию, вывод в десятичной системе счисления cout.unsetf(ios::dec); //Установить вывод в восьмеричной системе счисления	p=27

		<pre>cout. setf (ios :: oct); cout<<"p="<<p<<endl;</pre>	
hex	Вывод величин в шестнадцатеричной системе счисления	<pre>int p=23; //Отменить, установленный по умолчанию, вывод в десятичной системе счисления cout.unsetf(ios :: dec); //Установить вывод в шестнадцатеричной системе счисления cout. setf (ios :: hex); cout<<"p="<<p<<endl;</pre>	p=17
showbase	Выводить индикатор основания системы счисления	<pre>int p=23; cout.unsetf(ios :: dec); cout. setf (ios :: hex ios :: showbase); cout<<"p="<<p<<endl;</pre>	p=0x17
uppercase	Использовать прописные буквы в шестнадцатеричных цифрах	<pre>int p=29; cout.unsetf(ios :: dec); cout. setf (ios :: hex ios :: uppercase); cout<<"p="<<p<<endl;</pre>	p=1D
showpos	Выводить знак "+" для положительных чисел	<pre>int p=29; cout. setf (ios :: showpos); cout<<"p="<<p<<endl;</pre>	p=+29
scientific	Экспоненциальная форма вывода вещественных чисел	<pre>double p=146.673; cout. setf (ios :: scientific); cout<<"p="<<p<<endl;</pre>	p=1.466730e+002
fixed	Фиксированная форма вывода вещественных чисел (по умолчанию)	<pre>double p=146.673; cout. setf (ios :: fixed); cout<<"p="<<p<<endl;</pre>	p=146.673

Флаги удобно использовать в тех случаях, когда следует изменить параметры всех последующих операторов ввода-вывода. Использование большого количества флагов для управления одним или несколькими операторами ввода-вывода не совсем удобно, потом все установленные флаги придётся отключать.

Ещё одним способом форматирования является использование манипуляторов непосредственно в конструкциях cin и cout.

Использование манипуляторов форматирования

Манипуляторы встраиваются непосредственно в операторы ввода-вывода. В таблице ниже приведены основные манипуляторы форматирования с примерами их использования. Для корректного использования всех манипуляторов необходимо подключить библиотеку:

```
#include <iomanip>
```

Манипулятор	Описание	Пример использования	Результат
setw(n)	Определяет ширину поля вывода в n символов	int r=253; cout.setf (ios :: fixed); cout<<"r="<<setw(8)<<r<<endl;	r= 253
setprecision(n)	Определяет количество цифр $(n - 1)$ в дробной части числа	double h=1234.6578; cout.setf (ios :: fixed); cout<<"h="<<setw(15); cout<<setprecision(3); cout<<h<<endl;	h=1234.658
dec	Перевод числа в десятичную систему (по умолчанию)	int r=0253; cout<<"r="<<dec<<r<<endl;	; r=171
oct	Перевод числа в восьмеричную систему	int r=253; cout<<"r="<<oct<<r<<endl;	r=375
hex	Перевод числа в шестнадцатеричную систему	int r=253; cout<<"r="<<hex<<r<<endl;	p=fd
right	Выравнивание по правой границе	int r=-25; cout.width(15); cout<<"r="<<setw(15)<<right; cout<<r<<endl;	r=-25
left	Выравнивание по левой границе (по умолчанию)	int r=-25; cout.width(15); cout<<"r="<<setw(15)<<left; cout<<r<<endl;	r=-25
boolalpha	Вывод логических величин в текстовом виде (true, false)	bool a=true; cout<<boolalpha<<a<<endl;	true

noboolalpha	Вывод логических величин в числовом виде (1, 0)	bool a=true; cout<<noboolalpha<<a<<endl;	1
showpos	Выводить знак "+" для положительных чисел	int p=29; cout<<"p="<<showpos<<p<<endl;	p=+29
noshowpos	Не выводить знак "+" для положительных чисел	int p=29; cout<<"p="<<noshowpos<<p<<endl;	p=29
uppercase	Использовать прописные буквы в шестнадцатеричных цифрах	int p=253; cout<<"p="<<hex<<uppercase<<p<<endl;	p=FD
nouppercase	Использовать строчные буквы в шестнадцатеричных цифрах	int p=253; cout<<"p="<<hex<<nouppercase; cout<<p<<endl;	p=fd
showbase	Выводить индикатор основания системы счисления	int p=253; cout<<"p="<<hex<<uppercase<<showbase<<p<<endl;	p=0XFD
noshowbase	Не выводить индикатор основания системы счисления	int p=253; cout<<"p="<<hex<<uppercase; cout<<noshowbase<<p<<endl;	p=FD
setfill(c)	Установить символ c как заполнитель	cout<<"x="<<right<<setw(10)<<setprecision(4); cout<<setfill("!"); cout<<(float)1/7<<endl; cout<<"x="<<left<<setw(10); cout<<setprecision(4); cout<<setfill("!"); cout<<(float)1/7<<endl;	x=!!!!0.1429 x=0.1429!!!!
scientific	Экспоненциальная форма вывода вещественных чисел	double p=146.673; cout<<"p="<<scientific<<p<<endl;	p=1.466730e+002
fixed	Фиксированная форма вывода вещественных чисел (по умолчанию)	cout<<"p="<<fixed<<p<<endl;	p=146.673

Кроме того, управлять шириной поля вывода можно с помощью операторов:

- `cout.width(n)` — устанавливает ширину поля вывода — `n` позиций;
- `cout.precision(m)` — определяет `m` цифр в дробной части числа.

При использовании конструкций `cin` и `cout` фактически происходит ввод-вывод в текстовый файл. При вводе текстовым файлом является клавиатура ПК, при выводе в качестве текстового файла выступает экран дисплея, `cin` и `cout` фактически являются именами потоков, которые отвечают за ввод и вывод в текстовый файл. Поэтому многие рассмотренные возможности форматированного ввода-вывода будут использоваться и при обработке текстовых файлов.

Существует два основных типа файлов: текстовые и двоичные. Файлы позволяют пользователю считывать большие объёмы данных непосредственно с диска, не вводя их с клавиатуры.

Работа с текстовыми файлами в C++

Текстовыми называют файлы, состоящие из любых символов. Они организуются *по* строкам, каждая из которых заканчивается символом "конец строки". Конец самого файла обозначается символом "*конец файла*". При записи информации в текстовый *файл*, просмотреть который можно с помощью любого текстового редактора, все данные преобразуются к символьному типу и хранятся в символьном виде.

Для работы с файлами используются специальные типы данных, называемые потоками⁴. *Поток* `ifstream` служит для работы с файлами в режиме чтения. *Поток* `ofstream` служит для работы с файлами в режиме записи. Для работы с файлами в режиме как чтения, так и записи служит *поток* `fstream`.

В программах на C++ при работе с текстовыми файлами необходимо подключать библиотеки `iostream` и `fstream`.

Для того, чтобы записать данные в текстовый *файл*, необходимо:

1. Описать переменную типа `ofstream`.
2. Открыть файл с помощью функции `open`.
3. Вывести информацию в файл.
4. Закрыть файл.

Для того, чтобы считать данные из текстового файла, необходимо:

1. Описать переменную типа `ifstream`.
2. Открыть файл с помощью функции `open`.
3. Считать информацию из файла, при считывании каждой порции данных необходимо проверять достигнут ли конец файла.
4. Закрыть файл.

Запись информации в текстовый файл

(010_write_text.cpp)

Для того, чтобы начать работать с текстовым файлом, необходимо описать переменную типа `ofstream`. Например, с помощью оператора

```
ofstream F;
```

будет создана переменная `F` для записи информации в файл. На следующем этапе файл необходимо открыть для записи. В общем случае оператор открытия потока будет иметь вид:

```
F.open("file", mode);
```

Здесь `F` — переменная, описанная как `ofstream`, `file` — имя файла на диске, `mode` — режим работы с открываемым файлом.

Файл может быть открыт в одном из следующих режимов:

- **`ios::in`** — открыть файл в режиме чтения данных, этот режим является режимом по умолчанию для потоков `ifstream`;
- **`ios::out`** — открыть файл в режиме записи данных (при этом информация в существующем файле уничтожается), этот режим является режимом по умолчанию для потоков `ofstream`;
- **`ios::app`** — открыть файл в режиме записи данных в конец файла;
- **`ios::ate`** — передвинуться в конец уже открытого файла;
- **`ios::trunc`** — очистить файл, это же происходит в режиме `ios::out`;
- **`ios::nocreate`** — не выполнять операцию открытия файла, если он не существует⁶;
- **`ios::noreplace`** — не открывать существующий файл.

Параметр `mode` может отсутствовать, в этом случае файл открывается в режиме по умолчанию для данного потока⁷.

После удачного открытия файла (в любом режиме) в переменной `F` будет храниться `true`, в противном случае `false`. Это позволит проверять корректность операции открытия файла.

Открыть файл (в качестве примера возьмём файл `abc.txt`) в режиме записи можно одним из следующих способов:

```
//Первый способ.
ofstream f;
f.open ( " abc.txt ", ios::out );
//Второй способ,
//режим ios::out является режимом по умолчанию для потока ofstream
ofstream f;
f.open ( " abc.txt " )
//Третий способ объединяет описание переменной типа поток
//и открытие файла в одном операторе
ofstream f( " abc.txt ", ios::out );
```

После открытия файла в режиме записи будет создан пустой файл, в который можно будет записывать информацию. Если нужно открыть файл, чтобы в него что-либо дописать, то в качестве режима следует использовать значение `ios::app`.

После открытия файла в режиме записи, в него можно писать точно также, как и на экран, только вместо стандартного устройства вывода `cout` необходимо указать имя открытого для записи файла.

Например, для записи в поток `F` переменной `a` оператор вывода будет иметь вид:

```
F<<a;
```

Для последовательного вывода в поток `G` переменных `b`, `c` и `d` оператор вывода станет таким:

```
G<<b<<c<<d;
```

Закрытие потока осуществляется с помощью оператора:

```
F.close();
```

В качестве примера рассмотрим следующую задачу.

Задача: Создать текстовый файл `abc.txt` и записать туда `n` вещественных чисел.

Текст программы с комментариями:

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {
    int i, n;
    double a;
    ofstream f; //Описывает поток для записи данных в файл.
    f.open(" abc.txt "); //Открываем файл в режиме записи,
    //режим ios::out устанавливается по умолчанию.
    cout << " n = ";
    cin >> n; //Ввод количества вещественных чисел.
    for (i = 0; i < n; i++) {
        cout << " a = ";
        cin >> a; //Ввод очередного числа.
        if (i < n - 1) //Если число не последнее,
            f << a << " \t "; //записать в файл это число и символ табуляции, иначе
        else f << a; //записать только число.
    }
    f.close(); //Закрытие потока.
    return 0;
}
```

Обратите внимание, что в текстовый файл записываются не только вещественные числа, но и символы табуляции. Таким образом, в конце файла после последнего числа находится символ табуляции. По этой причине может возникнуть проблема при чтении информации, так как символ табуляции будет интерпретирован как вещественное число. Чтобы этого избежать в программе была применена следующая конструкция:

```
if (i<n-1)
    f<<a<<"\t";
else
    f<<a;
```

Здесь реализована проверка ввода последнего числа. После него символ табуляции отсутствует.

В результате работы программы будет создан текстовый файл abc.txt, который можно просмотреть средствами обычного текстового редактора.

```
5      2      88      55      33      22      67      34
```

Чтение информации из текстового файла

(011_read_text.cpp)

Для того, чтобы прочитать информацию из текстового файла, необходимо описать переменную типа **ifstream**. После этого файл необходимо открыть для чтения с помощью оператора **open**. Если переменную назвать **F**, то первые два оператора будут такими:

```
ifstream F;
F.open("abc.txt", ios::in);
```

После открытия файла в режиме чтения, из него можно считывать информацию точно так же, как и с клавиатуры, только вместо стандартного устройства ввода **cin** необходимо указать имя потока, из которого будет происходить чтение данных.

Например, для чтения из потока **F** в переменную **a** оператор ввода будет иметь вид:

```
F>>a;
```

Для последовательного ввода из потока **G** в переменные **b**, **c** и **d** оператор ввода станет таким:

```
G>>b>>c>>d;
```

Два числа в текстовом файле считаются разделёнными, если между ними есть хотя бы один из символов: пробел, табуляция, символ конца строки.

Хорошо, если программисту заранее известно, сколько и каких значений хранится в текстовом файле. Однако часто просто известен тип значений, хранящихся в файле, при этом количество значений в файле может быть различным. При решении подобной проблемы необходимо считывать значения из файла по одному, а перед каждым считыванием проверять, достигнут ли конец файла. Для проверки, достигнут или нет конец файла, служит функция

```
f.eof();
```

Здесь **F** — имя потока, функция возвращает логическое значение: **true** — если достигнут конец файла, если не достигнут функция возвращает значение **false**.

Следовательно, цикл для чтения содержимого всего файла можно записать так.

```
while ( ! f.eof ( ) ) //Организован цикл, условием окончания цикла
//является достижение конца файла, в этом случае f.eof() вернёт true.
{
    F>>a; //Чтение очередного значения из потока F в переменную a.
    ...
    <обработка значения переменной a>
}
```

Рассмотрим следующую задачу.

В текстовом файле **abc.txt** хранятся вещественные числа вывести их на экран и вычислить их количество.

Текст программы с комментариями приведён ниже.

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {
    ifstream f; //Поток для чтения.
    float a;
    int n = 0;
    f.open(" abc.txt "); //Открываем файл в режиме чтения.
    if (f) //Если открытие файла прошло корректно, то
    {
        while (!f.eof()) //Организован цикл, выполнение цикла
            //прервётся, когда будет достигнут конца файла.
        {
            f >> a; //Чтение очередного значения из потока f в переменную a.
            cout << a << " \t "; //Вывод значения переменной a
            n++; //Увеличение количества считанных чисел.
        }
        f.close(); //Закрытие потока.
        cout << " n = " << n << endl; //Вывод на экран количества чисел.
    } else cout << "файл не найден" << endl; //Если открытие файла прошло некорректно, то
    //вывод сообщения, об отсутствии такого файла.
    return 0;
}
```

Результат работы программы к задаче:

```
3.14159 2.789 -21.14 543.89 -90.1 n=5
```

Программа работает корректно, если текстовый файл abc.txt был создан с помощью программы из раздела про запись информации в текстовый файл. Предположим, что файл создавался в текстовом редакторе, и пользователь ввёл после последней цифры символ пробела, табуляции или перехода на новую строку. Тогда результат будет таким:

```
3.14159 2.789 -21.14 543.89 -90.1 -90.1 n= 6
```

Происходит это потому, что после чтения последней цифры из потока конец файла не достигнут, оператор цикла выполняется ещё один раз, значение переменной **n** увеличивается на единицу, а так как значение

переменной **a** не изменилось, то выводится повторно. Один из способов решения данной проблемы может быть таким:

```
while ( !f.eof ( ) )
{
    f>>a;
    if ( !f.eof ( ) ) //Проверяем, достигнут ли конец файла, если нет — печатаем значение a
    {
        cout<<a<<" \t ";
        n++;
    }
}
```

Если количество вещественных чисел, записанных в файл, известно заранее, то текст программы можно переписать следующим образом:

```
#include <iostream>
#include <fstream>
using namespace std;
int main ( )
{
    ifstream f;
    float a; int i, n=5;
    f.open ( " abc.txt " );
    if ( f )
    {
        for ( i =1; i<=n; f>>a, cout<<a<<" \t ", i++);
        f.close ( );
    }
    else cout<<"Файл не найден"<<endl;
    return 0;
}
```

Существует возможность открывать файл с данными таким образом, чтобы в него можно было дописывать информацию. Рассмотрим эту возможность на примере решения следующей задачи.

В файле **abc.txt** хранится массив вещественных чисел, дописать в файл этот же массив, упорядочив его по возрастанию.

Алгоритм решения задачи очень простой. Считываем в массив данные из текстового файла, упорядочиваем массив, дописываем его в этот же файл.

Для чтения данных из файла описываем поток **ifstream**, открываем его в режиме чтения и последовательно, пока не достигнем конца файла, считываем все элементы в массив. Сортировку проведём методом пузырька. Обратите внимание, что поток нужно открыть так, чтобы была возможность дописать в конец файла упорядоченный массив.

(012_sort_text.cpp)

Текст программы с комментариями приведён ниже.

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {
    ifstream f; //Поток для чтения.
    ofstream g; //Поток для записи.
    float *a, b;
    a = new float[100];
    int i, j, n = 0;
    f.open(" abc.txt ", ios::in); //Открываем файл в режиме чтения.
    if (f) {
        while (!f.eof()) {
            f >> a[n];
            n++;
        }
    }
}
```

```

    }
    //Сортировка массива.
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) {
                b = a[j];
                a[j] = a[j + 1];
                a[j + 1] = b;
            }
    f.close(); //Закрываем поток для чтения.
    g.open(" abc.txt ", ios::app); //Открываем поток для того, чтобы дописать данные.
    g << " \n "; //Запись в файл символа конца строки
    for (i = 0; i < n; i++) //Запись в файл
        if (i < n - 1) g << a[i] << " \t "; //элемента массива и символа табуляции.
        else g << a[i]; //Запись последнего элемента массива
    g.close(); //Закрытие файла.
} else cout << "Файл не найден" << endl;
delete[] a;
return 0;
}

```

Содержимое файла abc.txt после запуска программы

```

3.14159 2.798 -21.14 543.89 -90.1
-90.1 -21.14 2.798 3.14159 543.89

```

Обработка двоичных файлов

Если в файле хранятся только числа или данные определённой структуры, то для хранения таких значений удобно пользоваться двоичными файлами. В двоичных файлах *информация* считывается и записывается в виде блоков определённого размера, в них могут храниться данные любого вида и структуры.

Порядок работы с двоичными и текстовыми файлами аналогичен. Для того, чтобы записать данные в двоичный *файл*, необходимо:

1. Описать файловую переменную с помощью оператора **FILE *filename**; Здесь **filename** — имя переменной, где будет храниться указатель на файл.
2. Открыть файл с помощью функции **fopen**.
3. Записать информацию в файл с помощью функции **fwrite**.
4. Закрыть файл с помощью функции **fclose**.

Для того, чтобы считывать данные из двоичного файла, необходимо:

1. Описать переменную типа **FILE ***.
2. Открыть файл с помощью функции **fopen**.
3. Считать необходимую информацию из файла с помощью функции **fread**, при считывании информации следить за тем, достигнут ли конец файла.
4. Закрыть файл с помощью функции **fclose**.

Рассмотрим основные функции, необходимые для работы с двоичными файлами.

Для открытия файла предназначена *функция*:

FILE *fopen(const *filename, const char *mode); где, **filename** — строка, в которой хранится *полное имя* открываемого файла, **mode** — строка, которая определяет режим работы с файлом; возможны следующие значения:

- **"rb"** — открыть двоичный файл в режиме чтения;

- **"wb"** — создать двоичный файл для записи, если файл существует, то его содержимое очищается.
- **"ab"** — создать или открыть двоичный файл для добавления информации в конец файла;
- **"rb+"** — открыть существующий двоичный файл в режиме чтения и записи;
- **"wb+"** — открыть двоичный файл в режиме чтения и записи, существующий файл очищается;
- **"ab+"** — двоичный файл открывается или создаётся для исправления существующей информации и добавления новой в конец файла.

Функция **fopen** возвращает в файловой переменной **NULL** в случае неудачного открытия файла.

После открытия файла, в указателе содержится *адрес* 0-го байта файла. По мере чтения или записи *значение* указателя смещается на считанное (записанное) количество *байт*.

Текущее *значение* указателя — номер байта, начиная с которого будет происходить операция чтения или записи.

Для закрытия файла предназначена *функция*

```
int fclose(FILE *filename);
```

Она возвращает 0 при успешном закрытии файла и **NULL** в противном случае.

Для удаления файлов существует *функция*

```
int remove(const char *filename);
```

Эта *функция* удаляет с диска *файл* с именем **filename**. Удаляемый *файл* должен быть закрыт. *Функция* возвращает ненулевое *значение*, если *файл* не удалось удалить.

Для переименования файлов предназначена *функция*

```
int rename(const char *oldfilename, const char *newfilename);
```

здесь первый *параметр* — старое *имя файла*, второй — новое. Возвращает 0 при удачном завершении программы.

Чтение из двоичного файла осуществляется с помощью функции

```
fread (void *ptr, size, n, FILE *filename)
```

Эта *функция* считывает из файла **filename** в массив **ptr** **n** элементов размера **size**. *Функция* возвращает количество считанных элементов. После чтения из файла *указатель* файла смещается на **n*size** *байт*.

Запись в двоичный *файл* осуществляется с помощью функции

```
fwrite (const void *ptr, size, n, FILE *filename);
```

Функция записывает в *файл* filename из массива ptr n элементов размера size. *Функция* возвращает количество записанных элементов. После записи информации в *файл*, *указатель* файла смещается на n*size *байт*.

Для контроля достижения конца файла есть *функция*

```
int feof(FILE * filename);
```

Она возвращает ненулевое *значение*, если достигнут *конец файла*.

Рассмотрим использование двоичных файлов на примере решения двух стандартных задач.

Задача 1: Создать двоичный *файл* **abc.dat**, куда записать *целое число* **n** и **n** вещественных чисел.

```
#include <iostream>
#include <fstream>
using namespace std;
int main ( )
{
```

```

FILE * f; //Описание файловой переменной.
int i, n; double a;
f=fopen ( " abc.dat ", " wb " ); //Создание двоичного файла в режиме записи.
cout<<" n = "; cin>>n; //Ввод числа n.
fwrite ( &n, sizeof ( int ), 1, f ); //Запись числа в двоичный файл.
for ( i =0; i<n; i++) //Цикл для ввода n вещественных чисел.
{
    cout<<" a = "; cin>>a; //Ввод очередного вещественного числа.
    fwrite ( &a, sizeof ( double ), 1, f ); //Запись числа в двоичный файл.
}
fclose ( f ); //Закрыть файл.
return 0;
}

```

Задача 2: Вывести на экран содержимое созданного в предыдущей задаче двоичного файла abc.dat.

```

#include <iostream>
#include <fstream>
using namespace std;
int main ( )
{
    FILE * f; //Описание файловой переменной.
    int i, n; double *a;
    f=fopen ( " abc.dat ", " rb " ); //Открыть существующий двоичный файл в режиме чтения.
    fread ( &n, sizeof ( int ), 1, f ); //Читать из файла целое число в переменную n.
    cout<<" n = "<<n<<" \n "; //Вывод n на экран.
    a=new double [ n ]; //Выделение памяти для массива из n чисел.
    fread ( a, sizeof ( double ), n, f ); //Чтение n вещественных чисел из файла в массив a.
    for ( i =0; i<n; cout<<a [ i ]<<" \t ", i++); //Вывод массива на экран.
    cout<<endl;
    fclose ( f ); //Закрыть файл.
    return 0;
}

```

Двоичный *файл* — последовательная *структура данных*, после открытия файла доступен первый *байт*. Можно последовательно считывать из файла или записывать их в него. Допустим, необходимо считать пятнадцатое, а затем первое число, хранящееся в файле. С помощью последовательного доступа это можно сделать следующим образом.

```

FILE * f;
int i, n; double a;
f=fopen ( " file.dat ", " rb " );
for ( i =0; i <15; fread ( &a, sizeof ( double ), 1, f ), i++);
fclose ( f );
f=fopen ( " file.dat ", " rb " );
fread ( &a, sizeof ( double ), 1, f );
fclose ( f );

```

Как видно, такое чтение чисел из файла, а затем повторное *открытие файла* — не самый удачный способ. Гораздо удобнее использовать функцию перемещения указателя файла к заданному байту:

```
int fseek(FILE *F, long int offset, int origin);
```

Функция устанавливает *указатель* текущей позиции файла **F**, в соответствии со значениями начала отсчёта **origin** и смещения **offset**. *Параметр offset* равен количеству байтов, на которые будет смещён *указатель* файла относительно начала отсчёта, заданного параметром **origin**.

Если *значение offset* положительно, то *указатель* файла смещается вперёд, если отрицательно — назад. *Параметр origin* должен принимать одно из следующих значений, определённых в заголовке **stdio.h**:

- **SEEK_SET** — отсчёт смещения *offset* вести с начала файла;
- **SEEK_CUR** — отсчёт смещения *offset* вести с текущей позиции файла;
- **SEEK_END** — отсчёт смещения *offset* вести с конца файла.

Функция возвращает нулевое *значение* при успешном выполнении *операции* и ненулевое, если возник сбой при выполнении смещения.

Функция **fseek** фактически реализует *прямой доступ* к любому значению в файле. Необходимо только знать месторасположение (номер байта) значения в файле. Рассмотрим использование прямого доступа в двоичных файлах на примере решения следующей задачи.

В созданном двоичном файле **abc.dat** поменять местами наибольшее и наименьшее из вещественных чисел.

Алгоритм решения задачи состоит из следующих этапов:

- Чтение вещественных чисел из файла в массив **a**.
- Поиск в массиве **a** максимального (**max**) и минимального (**min**) значения и их номеров (**imax**, **imin**).
- Перемещение указателя файла к максимальному значению и запись **min**.
- Перемещение указателя файла к минимальному значению и запись **max**.

Ниже приведён текст программы решения задачи с комментариями.

```
#include <iostream>
#include <fstream>
using namespace std;
int main ( )
{
    FILE * f; //Описание файловой переменной.
    int i, n, imax, imin;
    double *a, max, min;
    f=fopen ( " abc.dat ", " rb + " ); //Открыть файл в режиме чтения и записи.
    fread (&n, sizeof ( int ), 1, f ); //Считать из файла в переменную n количество элементов
    в
    файле.
    a=new double [ n ]; //Выделить память для хранения вещественных чисел,
    //эти числа будут храниться в массиве a.
    fread ( a, sizeof ( double ), n, f ); //Считать из файла в массив a вещественные числа.
    //Поиск максимального, минимального элемента в массиве a, и их индексов.
    for ( imax=imin =0, max=min=a [ 0 ], i =1; i<n; i++)
    {
        if ( a [ i ]>max)
        {
            max=a [ i ];
            imax= i;
        }
        if ( a [ i ]<min )
        {
            min=a [ i ];
            imin= i;
        }
    }
    //Перемещение указателя к максимальному элементу.
    fseek ( f, sizeof ( int )+imax* sizeof ( double ),SEEK_SET);
    //Запись min вместо максимального элемента файла.
    fwrite (&min, sizeof ( double ), 1, f );
    //Перемещение указателя к минимальному элементу.
    fseek ( f, sizeof ( int )+imin * sizeof ( double ),SEEK_SET);
    //Запись max вместо минимального элемента файла.
    fwrite (&max, sizeof ( double ), 1, f );
    //Закрытие файла.
    fclose ( f );
    //Освобождение памяти, выделенной под массив a.
    delete [ ] a;
    return 0;
}
```